

AUTUMN 2026
Krutaya Komanda

Project D: Lexical Analysis

Where the lexer fits

01 SOURCE CODE

Contains keywords, identifiers, literals and operators.
Example: var total := 12.5

02 LEXER

Java, hand-written scanner.
Skips whitespace and comments.
Recognizes lexemes and emits typed tokens with source positions.
Characters → Tokens

03 PARSER + AST

Hand-written recursive-descent parser.
Checks whether tokens follow the language grammar.
Builds an abstract syntax tree.
Tokens → AST

04 INTERPRETER

Evaluates the AST directly.
Manages variables, values, functions and control flow.
Prints the program result.
AST → Result

Source code → Lexer → Tokens → Parser + AST → Interpreter

Transitioning from characters to tokens

PARSING INDIVIDUAL CHARACTERS

Input: original program text

Example: ``var total := 12.5``

CREATING LEXEMES

Organizing characters into meaningful groups

Example: ``var` · `total` · `:=` · `12.5``

GENERATING TOKENS

Assigning token types and values

Example: ``KW_VAR` · `IDENT(total)` · `ASSIGN` ·
`REAL(12.5)``

1

2

3

Token types in Project D

01 KEYWORDS

Reserved words that define program structure.
var · if · loop · func · return · print

02 IDENTIFIERS

Names created by the programmer.
total · grid · sumRow · packet

03 NUMERIC LITERALS

Integer and real values.
12 · 12.5

04 OTHER LITERALS

Strings, booleans and the none value.
"demo" · true · false · none

05 OPERATORS

Assignment, arithmetic and comparison. := · + · <= · => · ..

06 DELIMITERS

Symbols that structure expressions and collections.
() · [] · {} · , · · · ;

LEXER TRACE

Simple example: `var total := 12.5`

Source fragment	Scanner rule	Token type	Produced token
<code>var</code>	read letters, check keyword table	keyword	<code>KW_VAR</code>
<code> </code>	whitespace	skip	—
<code>total</code>	read letters	identifier	<code>IDENT(total)</code>
<code>:=</code>	longest match	assignment operator	<code>ASSIGN</code>
<code>12.5</code>	read digits and decimal point	real literal	<code>REAL(12.5)</code>

Source code

```
var add := func(a, b) => a + b
var multiply := func(a, b) => a * b

var calculator := {
  left := 12,
  right := 30,
  operations := {
    add := add,
    multiply := multiply
  }
}

var apply := func(op, input) =>
  op(input.left, input.right)

print apply(calculator.operations.add, calculator)
print apply(calculator.operations.multiply, calculator)
```

EXAMPLE 1: CALCULATOR

FUNCTION VALUE

`func(a, b) => a + b`
KW_FUNC - LPAREN - IDENT - COMMA - IDENT
- RPAREN - FAT_ARROW - IDENT - PLUS - IDENT

NESTED TUPLE ACCESS

`calculator.operations.add`
IDENT - DOT - IDENT - DOT - IDENT

FUNCTION CALL

`apply(op, calculator)`
IDENT - LPAREN - IDENT - COMMA - IDENT -
RPAREN

Source code

```
var grid := [[1, 2], [3, 4]]

var packet := {
  meta := {id := 7},
  rows := grid,
  tags := ["lexer", "demo"]
}

var apply := func(f, value) => f(value)
var rowSum := func(row) => row[1] + row[2]

print apply(rowSum, packet.rows[2])
```

EXAMPLE 2: NESTED VALUES

ARRAY OF ARRAYS

`[[1, 2], [3, 4]]`

LBRACKET - LBRACKET - INTEGER - COMMA -
INTEGER - RBRACKET - COMMA - ...

NESTED TUPLE

`meta := {id := 7}`

IDENT - ASSIGN - LBRACE - IDENT - ASSIGN -
INTEGER - RBRACE

FUNCTION + CHAINED REFERENCE

`apply(rowSum, packet.rows[2])`

IDENT - LPAREN - IDENT - COMMA - IDENT -
DOT - IDENT - LBRACKET - INTEGER -
RBRACKET - RPAREN